

Практикум 2. Работа с требованиями и диаграммами

Цель: Выбрать ИИ-проект для дальнейших практических работ и подготовить его базовую документацию: комплект документов, User Stories, Use Case и UML-диаграммы.

Выбранный проект будет использоваться в следующих практических работах курса.

Что понадобится

- GitHub-репозиторий из Практикума 1;
- MkDocs или Docusaurus;
- Markdown;
- Mermaid или аналог для создания диаграмм.

Шаг 1. Выберите ИИ-проект

Выберите **один проект, использующий искусственный интеллект или машинное обучение**.

Этот проект станет сквозным кейсом для следующих практических работ, поэтому выбирайте достаточно простую и понятную идею.

Можно выбрать тему самостоятельно или использовать одну из предложенных.

Примеры тем

- **Классификатор обращений в техническую поддержку** — определяет категорию обращения и направляет его нужной группе специалистов.
- **Рекомендательная система фильмов** — предлагает фильмы на основе интересов пользователя.
- **ИИ-помощник для студентов** — отвечает на вопросы по учебным материалам.
- **Анализатор резюме** — извлекает навыки, образование и опыт из резюме.
- **Система определения спама** — классифицирует сообщения как обычные или нежелательные.
- **Анализатор отзывов** — определяет тональность отзыва: положительная, нейтральная или отрицательная.
- **Рекомендательная система учебных курсов** — предлагает студенту курсы на основе его интересов и пройденных дисциплин.
- **Классификатор изображений** — определяет категорию объекта на загруженной фотографии.
- **ИИ-помощник интернет-магазина** — отвечает на вопросы о товарах и помогает подобрать подходящий вариант.
- **Сервис автоматического краткого пересказа текста** — принимает большой текст и

формирует краткое содержание.

Можно предложить собственную тему.

Не выбирайте слишком большую систему. Для практических работ достаточно небольшого сервиса с **2–4 основными функциями**.

Шаг 2. Создайте раздел проекта

Продолжайте работать в репозитории из Практикума 1.

Создайте папку: `docs/project/`

В ней создайте файл: `overview.md`

Добавьте название и краткое описание проекта.

Например:

```
# Классификатор обращений
```

Сервис автоматически определяет категорию обращения пользователя и направляет его соответствующей группе технической поддержки.

Основные пользователи:

- пользователь;
- оператор поддержки.

Основные функции:

- отправка обращения;
- определение категории обращения;
- просмотр результата классификации.

Достаточно нескольких абзацев.

Шаг 3. Определите комплект документации

В `overview.md` добавьте раздел:

```
## Комплект документации
```

Определите, какая техническая документация потребуется вашему проекту.

Укажите **минимум 5 документов или разделов документации**.

Например:

1. Описание проекта
2. Требования
3. Архитектура системы
4. Документация API
5. Документация ML-модели
6. Руководство пользователя

Для каждого пункта одной фразой напишите, для кого он предназначен и зачем нужен.

Не нужно создавать все эти документы сейчас. На этом этапе вы только определяете будущий комплект.

Шаг 4. Создайте User Stories

Создайте файл:

docs/project/requirements.md

Опишите **3 User Stories** для своего проекта.

Используйте формат:

Как [роль],
я хочу [возможность],
чтобы [ценность].

Например:

Как оператор поддержки,
я хочу автоматически получать категорию обращения,
чтобы быстрее направлять его нужной группе.

Для каждой User Story добавьте **2–3 критерия приёмки**.

Например:

US-01. Классификация обращения

Как оператор поддержки,
я хочу автоматически получать категорию обращения,
чтобы быстрее направлять его нужной группе.

Критерии приёмки:

- система принимает текст обращения;
- после обработки отображается определённая категория;
- если классификация невозможна, пользователь получает сообщение об ошибке.

Не усложняйте требования. Важнее, чтобы они были понятными и проверяемыми.

Шаг 5. Создайте Use Case

В том же файле requirements.md опишите **один основной Use Case**.

Выберите ключевую функцию проекта.

Например:

«Классификация обращения».

Используйте структуру:

UC-01. Классификация обращения

Актор: оператор поддержки

Цель: определить категорию обращения.

Предусловие: пользователь отправил обращение.

Основной сценарий:

1. Оператор открывает обращение.
2. Система передаёт текст ML-модели.

3. ML-модель определяет категорию.
4. Система получает результат.
5. Оператор видит определённую категорию.

Альтернативный сценарий:

Если модель не может обработать текст, система сообщает, что требуется ручная классификация.

Результат: категория обращения определена или обращение передано на ручную обработку.

Для своего проекта используйте собственный сценарий.

Шаг 6. Создайте диаграмму последовательности

Теперь визуализируйте созданный Use Case.

Создайте файл: docs/project/diagrams.md

Добавьте раздел:

```
# Диаграммы
```

```
## Диаграмма последовательности
```

Создайте **Sequence Diagram** — диаграмму последовательности.

Она должна показывать взаимодействие пользователя и основных компонентов системы при выполнении выбранного Use Case.

Например:

```
sequenceDiagram
    actor User as Оператор
    participant System as Система
    participant Model as ML-модель

    User->>System: Отправить обращение
    System->>Model: Передать текст
    Model-->>System: Вернуть категорию
    System-->>User: Показать результат
```

Для своего проекта измените участников и действия.

Диаграмма должна содержать минимум:

- одного пользователя;
- систему или приложение;
- ML/AI-компонент;
- основные запросы и ответы.

Шаг 7. Создайте диаграмму классов

В том же файле добавьте:

```
## Диаграмма классов
```

Создайте простую **Class Diagram** — диаграмму классов.

Не нужно проектировать всю систему.

Достаточно **3–5 основных сущностей**.

Например:

```
classDiagram
```

```
class User {
    +int id
    +string name
}

class SupportRequest {
    +int id
    +string text
    +string category
}

class ClassificationResult {
    +string category
    +float confidence
}

User "1" --> "*" SupportRequest
SupportRequest "1" --> "1" ClassificationResult
```

Для своего проекта определите собственные сущности и связи.

Например, для рекомендательной системы это могут быть:

```
User;
```

```
Movie;
```

```
Recommendation;
```

```
UserPreference.
```

Задача этой работы — **попробовать описать структуру системы диаграммой**, а не построить идеальную архитектурную модель.

Шаг 8. Подключите Mermaid

Если ваш генератор документации уже отображает Mermaid-диаграммы, дополнительная настройка не требуется.

Если диаграммы отображаются как обычный текст, подключите поддержку Mermaid для используемого генератора документации.

Для MkDocs можно использовать Mermaid через расширение или соответствующий плагин/тему.

Для Docusaurus также доступна поддержка Mermaid.

После настройки перезапустите локальный сервер.

Шаг 9. Добавьте страницы в навигацию

Добавьте созданные страницы в меню документации.

Для MkDocs это можно сделать в `mkdocs.yml`.

Например:

```
nav:
  - Главная: index.md
  - Проект:
    - Описание: project/overview.md
    - Требования: project/requirements.md
    - Диаграммы: project/diagrams.md
```

Названия пунктов можно изменить под свой проект.

Шаг 10. Соберите и запустите документацию

Запустите документацию локально.

Для MkDocs: `mkdocs serve`

Для Docusaurus: `npm run start`

Откройте документационный сайт в браузере.

Проверьте:

- открывается описание проекта;
- отображаются User Stories;
- отображается Use Case;
- открывается диаграмма последовательности;
- открывается диаграмма классов;
- между страницами можно переходить через меню.

Шаг 11. Сохраните результат в Git

Добавьте изменения: `git add .`

Создайте commit: `git commit -m "Add project requirements and diagrams"`

Отправьте изменения: `git push`

Проверьте себя

Перед сдачей убедитесь, что:

- выбран один ИИ-проект;
- есть краткое описание проекта;

- определён комплект документации;
- создано 3 User Stories;
- для каждой User Story есть критерии приёмки;
- создан 1 Use Case;
- есть диаграмма последовательности;
- есть диаграмма классов;
- документация собирается и открывается локально;
- изменения загружены на GitHub.

Что сдавать

Пришлите преподавателю **одну ссылку на GitHub-репозиторий**.

Дополнительно добавьте в папку `screenshots` **один скриншот локально запущенного сайта** и два скриншота, на которых видны созданные диаграммы.

Итоговая структура может выглядеть так:

```
docs-as-code/
├── docs/
│   ├── index.md
│   └── project/
│       ├── overview.md
│       ├── requirements.md
│       └── diagrams.md
├── screenshots/
│   ├── site.png
│   └── practice-2.png
├── mkdocs.yml
└── README.md
```

Для сдачи отправьте:

Фамилия Имя — название ИИ-проекта — ссылка на GitHub-репозиторий.

Отдельный отчёт, презентацию или архив отправлять не нужно.

Критерии проверки

Зачёт, если:

- выбран ИИ-проект и понятно его назначение;
- определён комплект документации;
- есть 3 корректных User Stories;
- есть критерии приёмки;
- есть 1 понятный Use Case;
- обе диаграммы соответствуют выбранному проекту;
- документация собрана;
- результат находится в GitHub.

С этого практикума **тема проекта фиксируется**.

В следующих практических работах продолжайте работать с этим же ИИ-проектом и постепенно дополняйте его документацию.